Алгоритм

ШУТИС, Хэрэглээний Шинжлэх ухааны Сургууль, Я.Лутбат

- Алгоритм гэдэг бол асуудлыг шийдвэрлэхэд хийгдэх үйлдлүүдийн нарийн заагдсан зөвлөмжүүдийн төгсгөлөг дараалал юм.
- Алгоритм гэдэг үг нь тооны орчин цагийн аравтын бичлэгийн суурь болох Хиндү тооны тухай номын зохиогч, МЭ-ний 9-р зууны үеийн математикч *al-Khowarizmi-ийн* нэрнээс гаралтай юм.

- Төгсгөлөг тооны бүхэл тоонуудын дарааллаас максимумыг (хамгийн их) олох алгоритмыг тодорхойл.
- Дараах алхамуудыг хийх ёстой.
  1. Максимум утганд дарааллын эхний элементийг түр авна (алхам бүрт максимум утга гарч ирнэ).
  2. Дараалалд байх дараагийн бүхэл тоог түр сонгон авсан максимум утгатай харьцуулж үзэх ба хэрэв максимум утгаас их бол энэ тоог максимум утганд “түр” тооцно.
  3. Өмнөх алхамыг хэрэв дараалалд өөр бүхэл тоо үлдэж байвал цааш давтана.
  4. Дараалалд өөр тоо үлдэхгүй бол дээрх давталтыг зогсоох ба түр сонгон авсан максимум утга дарааллын хамгийн их элемент байх юм.

Жишээ

Псевдокод

- Алгоритмыг бид *псевдокод* ашиглан бичиж тэмдэглэх бөгөөд энэ нь ердийн ярианы хэлнээс програмчлалын тодорхой хэлрүү алгоритмыг буулгаж ирэх хоёрын завсрын үе шат юм.
- Бидний ашиглах псевдокод нь жишээнээс ойлгомжтой байх болно.
- Энд, ашиглагдах псевдокодны дэлгэрэнгүйг Kenneth Rosen H “Discrete mathematics and its application” номны хавсралт 3-аас харж болно.

ALGORITHM 1

procedure $max(a_1, a_2, \ldots, a_n$: integers)

$max := a_1$

for i:=2 **to** n

if $max < a_i$ **then** $max := a_i$

return max {max нь максимум элемент}

Төгсгөлөг дараалалын хамгийн их элементийг олох

Алгоритмын чанар

- *Оролт.* Алгоритм тодорхой олонлогоос оролтын утгуудтай байх ёстой.
- *Гаралт.* Оролтын утгууд тодорхой олонлогт гаралтын утгуудтай байх ёстой. Гаралтын утгууд нь тухайн бодлогын шийд байна.
- *Тодорхой байх.* Алгоритм хийгдэх үйлдлүүд нарийн тодорхойлогдсон байна.
- *Зөв байх.* Оролтын утга бүрт алгоритм зөв гаралтын утгыг харгалзуулдаг байна.
- *Төгсгөлөг алхмын дараа дуусдаг байх.* Алгоритм шаардлагатай үр дүнг төгсгөлөг тооны алхмын дараа өгдөг байна.
- *Эрчимтэй байх.* Алгоритмд зайлшгүй шаардлагатай тэр алхмууд төгсгөлөг хугацаанд хийгдэх ёстой.
- *Ерөнхий байх.* Зөвхөн нэг төрлийн оролтын утгуудад бус ижил төрлийн бүх бодлогуудад алгоритмийг хэрэглэж болдог байх ёстой.

Хайлтын Алгоритм

- Элементүүдийн эрэмбэлэгдсэн жагсаалтаас ямар нэг элементийн олох асуудал түгээмэл тааралдана.
- Жишээлбэл, аливаа үгийг зөв бичигдсэн эсэхийг шалгахдаа тухайн программ толь бичгээс уг үгийг хайж олох шаардлагатай болдог.
- Ийм төрлийн бодлогыг *хайлтын бодлого* гэдэг.
- Хайлтын бодлогыг ерөнхий тохиолдолд дараах байдлаар тодорхойлж болно:
  - Ялгаатай элементүүдийн $a_1, a_2, \ldots, a_n$ жагсаалтад хэрэв x элемент байгаа бол түүний байрлалыг ол, эсрэг тохиолдолд, жагсаалтад байхгүй гэдгийг нь тодорхойл. Хайлтын бодлогын үр дүнд x-тэй тэнцүү элементийн жагсаалт дах байрлал олдох ба хэрэв x жагсаалтад байхгүй бол байрлал нь 0 гэж гарч ирнэ.

ALGORITHM 2

procedure *linear search*(x: integer, $a_1, a_2, \ldots, a_n$: ялгаатай бүхэл тоонууд)

$i := 1$

while ($i \leq n$ and $x \neq a_i$)

$i := i + 1$

if $i \leq n$ **then** *location* := i

else *location* := 0

return *location*{*location* нь бол x-тэй тэнцүү элементийн дугаар, хэрэв x элемент жагсаалтаас олдохгүй бол 0 байна}

Шугаман Хайлт

Хоёртын хайлт

- Энэ алгоритм нь жагсаалтанд буй элементүүд өсөх дарааллаараа эрэмбэлэгдсэн тохиолдолд ашиглагдана.
- Жишээлбэл тоонууд багаас ихрүүгээ, үгнүүд цагаан толгойн эрэмбээр буюу толь бичгийн эрэмбээрээ эрэмбэлэгдсэн байна.
- Хоёртын хайлт нь жагсаалтын голын элементтэй жиших аргаар явагдана.
- Жагсаалт нь ижил урттай, эсвэл нэг нь нөгөөгөөс нэг элементээр цөөн хоёр жижиг жагсаалтад цааш хуваагдах ба
- голын элемент ба хайж буй элементийн жишилт дээр үндэслэн хайлт цааш тохирох жижиг жагсаалтруу үргэлжлэнэ.

Жишээ

1 2 3 5 6 7 8 10 12 13 15 16 17 19 20 22 жагсаалтаас 19-ийг ольё.

- 16 элементтэй энэ жагсаалтыг тус бүртээ 8 элемент бүхий хоёр жижиг жагсаалтад болгон хуваана. Энд,

  1 2 3 5 6 7 8 10 12 13 15 16 17 19 20 22.
- 19-ийг эхний жагсаалтын хамгийн их элементтэй харьцуулна.
- 10 < 19 тул, хайлтаа анхны жагсаалтын 9 дүгээр элементээс 16 дах элементийг агуулах хэсэг дээр цааш үргэлжлүүлнэ.
- Одоо 8 элемент бүхий сүүлийн жагсаалтыг тус бүртээ 4 элемент бүхий хоёр жижиг жагсаалтад болгон хуваана. Тухайлбал,

  12 13 15 16 17 19 20 22.
- 16 < 19 тул (19-ийг эхний жагсаалтын хамгийн их элементтэй харьцуулна) хайлтаа хоёр дах жагсаалт дээр үргэлжлүүлэх ба
- 18 19 20 22 жагсаалтыг дахин хоёр хэсэгт хуваана. Энд,

  17 19 20 22.
- Эхний жагсаалтын хамгийн их элементээс 19 их биш байна. Иймд 19-ийг эхний 17 19 жагсаалтаас цааш үргэлжүүлэн хайх шаардлагатай. Энэ жагсаалт анхны жагсаалтын 13, 14 дүгээр элементүүдийг агуулж.
- Эцэст нь, сүүлийн жагсаалтыг 17 ба 19 гэсэн тус бүртээ нэг нэг элементтэй бүхий хоёр жагсаалт болгон хуваана.
- 17 < 19 тул хайлтаа хоёр дах жагсаалт дээр үргэлжлүүлнэ. Энэ жагсаалт анхны жагсаалтын 14 дүгээр элемент буюу 19-ийг агуулна.
- Хайлт нэг элементтэй жагсаалт хүртэл үргэлжлэсэн мөн харьцуулалт хийгдсэн. Иймд 19 бол анхны жагсаалтын 14 дэх элемент байх болно.

Хоёртын хайлт

- x-ийн утгыг $a_1 < a_2 < \cdots < a_n$ байх $a_1, a_2, \ldots, a_n$ жагсаалтаас хайхдаа эхлээд x-ийн утгыг жагсаалтын дунд элемент a_m-тэй харьцуулна. Энд, $m = \lfloor (n+1)/2 \rfloor$ бөгөөд $\lfloor a \rfloor$ нь a тооноос үл хэтрэх хамгийн их бүхэл тоо юм.
- Хэрэв $x > a_m$ бол x-ийн хайлтыг жагсаалтын хоёр дах $a_{m+1}, a_{m+2}, \ldots, a_n$ хагаст үргэлжлүүлнэ.
- Хэрэв x нь a_m-ээс их биш бол x-ийн хайлтыг жагсаалтын эхний хагаст үргэлжлүүлнэ.
- Хайлтыг $\lceil n/2 \rceil$-оос олонгүй элемент бүхий жагсаалт дээр үргэлжлүүлэхээр болсон. Энд, $\lceil x \rceil$ нь x-ээс их буюу тэнцүү хамгийн бага бүхэл тоо юм.
- Өмнөхийн адилаар x-ийг жижиг жагсаалтын дунд элементтэй харьцуулж, хайлтыг жагсаалтын эхний эсвэл хоёр дахь хагаст үргэлжлүүлэх ба энэ процессийг жагсаалт ганц элементтэй болтол үргэлжлүүлнэ.
- Эцэст нь, үлдсэн ганц элемент x эсэхийг шалгана.

ALGORITHM 3

procedure *binary search* (x: integer, $a_1, a_2, \ldots, a_n$: бүхэл тооны өсөх дараалал)

$i := 1$ {i нь хайлтын интервалын зүүн төгсгөл}

$j := n$ {j нь хайлтын интервалын баруу төгсгөл}

while $i < j$

$m := (i + j)/2$

if $x > a_m$ **then** $i := m+1$

else $j := m$

if $x = a_i$ **then** *location* := i

else *location* := 0

return *location*{*location* нь x-тэй тэнцүү a_i элементийн дугаар i байх ба эсрэг тохиолдолд, хэрэв x нь анхны жагсаалтаас олдохгүй бол 0 байна}

Хоёртын хайлт

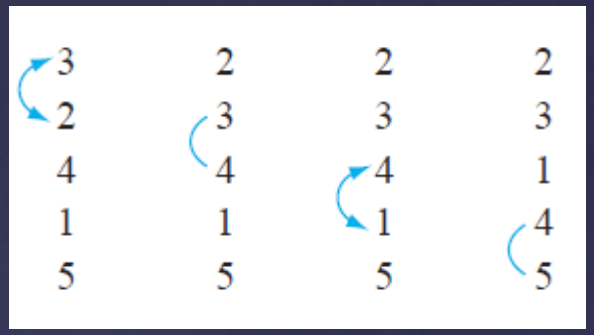
Эрэмбэлэх бодлого

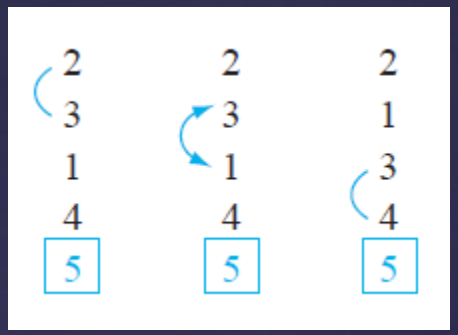
- Бидэнд элементүүдийн жагсаалт өгөгдсөн ба эрэмбэлэх арга нь мэдэгдэж байг. Тэгвэл энэ жагсаалтанд буй элементүүдийг өсөх эрэмбээр нь жагсаан бичиж болох бөгөөд үүнийг уг жагсаалтыг *эрэмбэлэх* гэнэ.
- Жишээлбэл, 7, 2, 1, 4, 5, 9 жагсаалтыг эрэмбэлэхэд шинэ 1, 2, 4, 5, 7, 9 жагсаалт үүснэ. Харин *d*, *h*, *c*, *a*, *f* жагсаалтыг эрэмбэлэвэл (энд, цагаан толгойн эрэмбийг ашиглалаа) *a*, *c*, *d*, *f*, *h* жагсаалт үүснэ.

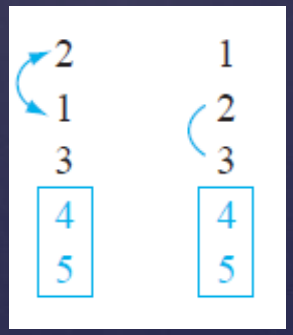
- Энэ арга бол эрэмбэлэх хамгийн хялбар алгоритмуудын нэг хэдий ч хамгийн эрчимтэй нь хараахан биш ээ
- Энэ аргад зэргэлдээ элементүүдийг харьцуулах ба хэрэв эдгээр элементүүдийн эрэмбэ алдагдсан бол тэдгээрийн байрыг сольж улмаар жагсаалтыг өсөх эрэмбээр нь бичдэг.
- Алгоритмыг хэрэгжүүлэхдээ дараах үндсэн үйлдлийг хийнэ. Үүнд,
  - жагсаалтын эхнээс “том” элементийг түүний “ард” байрлах илүү “жижиг” элементтэй байр солих ба энэ мэтчилэн жагсаалтыг дуустал үргэлжлүүлнэ.
- Бид энэ үндсэн үйлдлээ жагсаалтийг бүхэлдээ эрэмбэлэгдэж дуустал давтан хийнэ.
- Хэрэв бид жагсаалтыг багана болгож бичсэн гэж төсөөлвөл харьцуулж эрэмбэлэх аргаар жижиг “хөөснүүд” дээшлэн том хөөснүүдийн дээр аажмаар гарч ирэх бөгөөд том хөөснүүд доош “живнэ”.
- Энэ утгаар нь *bubble sort* эсвэл *sinking sort* гэж нэрлэх нь бий.

Харьцуулж эрэмбэлэх (Bubble sort)

- 3, 2, 4, 1, 5 жагсаалтыг өсөх эрэмбээр нь эрэмбэлье.











:байр сэлгэх үйлдэл

:зөв эрэмбэлэгдсэн хос

өнгөөр ялгасан тоонууд бол нэгэнт эрэмбэлэгдсэн жагсаалт

Жишээ

ALGORITHM 4

procedure *bubblesort*($a_1, \ldots, a_n$: real numbers with $n \geq 2$)

for $i := 1$ **to** $n - 1$

for $j := 1$ **to** $n - i$

if $a_j > a_j + 1$ **then** a_j *ба* a_{j+1} *элементүүдийн байрыг соль*

{$a_1, \ldots, a_n$ өсөх дарааллаараа эрэмбэлэгдсэн жагсаалт}

Харьцуулж эрэмбэлэх

Оруулж эрэмбэлэх (Insertion sort)

- Оруулж эрэмбэлэх арга бол эрэмбэлэх хялбар алгоритмуудын нэг юм. Гэвч, энэ арга ямагт хамгийн эрчимтэй алгоритм байж чадахгүй.
- Энэ алгоритм n элементтэй жагсаалтын 2 дугаар элементээс эхлэх ба уг элементийг жагсаалтын эхний элементтэй харьцуулж,
  - хэрэв эхний элементээс үл хэтрэх бол эхний элементийн өмнө нь оруулна.
  - Хэрэв эхний элементээс их бол энэ элементийн "ард" оруулна.
- Энэ үед эхний 2 элемент зөв эрэмбэлэгдсэн байх юм.
- Одоо 3 дах элементийг эхний элементтэй харьцуулна.
  - Хэрэв эхний элементээс их бол 2 дах элементтэй харьцуулна. Улмаар 3 дах элементийн хамт эхний гурван элемент зөв байрлалдаа орно.
- Ерөнхий тохиолдолд, j дүгээр алхамд жагсаалтын j дүгээр элемент өмнөх $j-1$ элементүүдийн урьд эрэмбэлэгдсэн жагсаалтанд зөв байрлалдаа орно.
- Уг жагсаалтанд j дүгээр элементийг оруулахдаа шугаман хайлтын аргын техникийг ашиглана.
  - Урьд өмнө эрэмбэлэгдсэн $j-1$ элементүүдийг эхнээс j дүгээр элементтэй харьцуулах ба j дүгээр элементээс бага биш элемент олдтол эсвэл бүх $j-1$ элементүүд жишигдэж дуустал үргэлжлүүлнэ.
- j элемент зөв байрлалдаа ороход эхний j элементүүд эрэмбэлэгдэж дуусна.
- Энэ алгоритмийг сүүлчийн элемент өмнө эрэмбэлэгдсэн $n-1$ элементийн жагсаалтанд зөв байрлалдаа орох хүртэл үргэлжлүүлнэ.

Жишээ

Оруулж эрэмбэлэх алгоритм ашиглаж 3, 2, 4, 1, 5 жагсаалтыг эрэмбэлье.

- Оруулж эрэмбэлэх аргаар эхлээд 2 ба 3 тоонуудыг харьцуулна. 3 > 2 учираас 2-ыг эхний байрлалд оруулна.

  2, 3, 4, 1, 5 (эрэмбэлэгдсэн хэсгийг өнгөөр ялгаж харуулаа).
- Дараагийн алхамд 4-ийг 2-той харьцуулах ба 4 > 2 тул 4-ийг 3-тай харьцуулна. 4 > 3 учраас 4 нь 3 дах байрлал дээр хэвээр үлдэнэ. Иймд,

  2, 3, 4, 1, 5 байх бөгөөд зөв эрэмбэтэй байна.
- Одоо дөрөв дэх элемент 1-ийг, өмнө эрэмбэлэгдсэн 2, 3, 4 жагсаалтанд зөв байрлалд нь оруулах ёстой. 1 < 2 тул,

  1, 2, 3, 4, 5 жагсаалт гарч ирнэ.
- Эцэст нь, 5-ыг 1, 2, 3, ба 4-тэй харьцуулснаар зөв байрлалд нь оруулж болно. 5 > 4 тул 5 жагсаалтын төгсгөлд хэвээр үлдэнэ.
- Энд гарч ирсэн жагсаалт бүхэлдээ зөв эрэмбэлэгдсэн жагсаалт байна.

ALGORITHM 5

procedure *оруулж эрэмбэлэх*($a_1, a_2, \ldots, a_n$: бодит тоонууд ба $n \geq 2$)

for $j := 2$ **to** n

$i := 1$

while $a_j > a_i$

$i := i + 1$

$m := a_j$

for $k := 0$ **to** $j - i - 1$

$a_{j-k} := a_{j-k-1}$

$a_i := m$

{$a_1, \ldots, a_n$ өсөх дарааллаараа эрэмбэлэгдсэн жагсаалт}

Оруулж эрэмбэлэх

Ховдог алгоритмууд (Greedy Algorithms)

- Олон тооны алгоритм *оновчлолын бодлого* бодоход зориулагдсан байдаг.
- Оновчлолын бодлого нь зарим төрлийн параметрүүдийн утгыг максимумчилах эсвэл минимумчилахад чиглэсэн байдаг. Жишээлбэл, хоёр хотын хоорондох хамгийн богино замыг олох, мэдээллийг хамгийн цөөн тооны битүүдээр кодлох гэх мэт бодлогууд нь оновчлолын бодлого юм.
- Оновчлолын бодлогын шийдийг олох (гэхдээ үргэлж олох албагүй) нэг хялбар техник бол
  - алхам бүртээ хамгийн сайн сонголтыг хийх
  - Улмаар оновчтой шийдийг олох явдал юм. Гэвч энэ бүх алхмыг хийж дуусахад оновчтой шийд олдсон байх албагүй.
- Алхам бүртээ “шилдэг”гэж үзэж буй тэр сонголтоо хийдэг алгоритмуудыг *greedy* буюу *ховдог алгоритм* гэнэ.
- Энэ төрлийн алгоритмаар олдсон шийд оновчтой шийд мөн эсэхийг заавал шалгах ёстой.

n центийг 25, 10, 5, ба 1 центийн зоосон мөнгөөр аль болох цөөн тооны зоос ашиглан солих тухай бодлогыг авч үзье.

- *n* центийг солихдоо алхам бүртээ хамгийн сайн гэж тооцогдох сонголтуудыг хийе.
  - Үүний тулд алхам бүрт *n*-ээс хэтрэхгүй байхаар хамгийн “том” зоосыг сонгож авъя.
- Жишээлбэл, 67 центийг солъё.
  - Эхлээд бид 25 центийн зоос сонгоно (одоо 42 цент солих ёстой).
  - Дараа нь, хоёр дах 25 центийн зоосыг сонгох хэрэгтэй (17 цент үлдсэн),
  - 10 цент сонгоно (7 цент үлдэнэ),
  - 5 цент сонгоно (2 цент үлдсэн),
  - 1 цент сонгоно (1 цент үлдэнэ),
  - дахин 1 цент сонгож авна.
- Эцэст нь, 67 цент солих боломжтой боллоо.

Жишээ

ALGORITHM 6

procedure *мөнгө солих*($c_1, c_2, \ldots, c_r$: зоосон мөнгө энд, $c_1 > c_2 > \cdots > c_r$; n: эерэг бүхэл тоо)

for i := 1 **to** r

d_i:=0 {d_i нь c_i зоосыг сонгож авсан тоо хэмжээ}

while $n \geq c_i$

d_i:=d_i+1 {add a coin of denomination ci}

n:=$n-c_i$

{d_i ба $i = 1, 2, \ldots, r$ нь n центийг солиход шаардагдах c_i зоосыг сонгож авсан тоо хэмжээ}

THEOREM Хамгийн цөөн тооны зоосон мөнгө бүхий мөнгө солилтыг Greedy алгоритмаар (Algorithm 6) хийж болно.

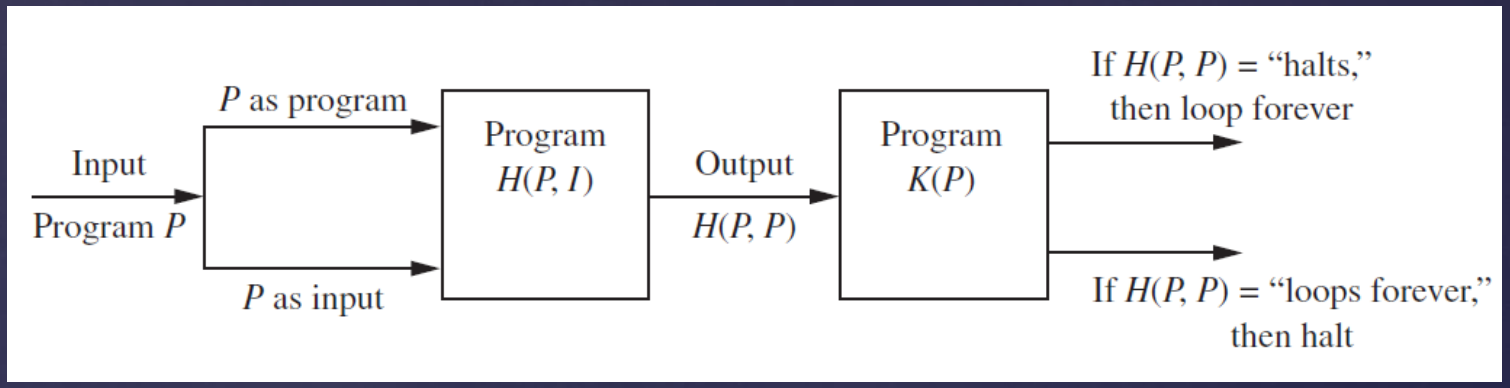
Greedy Algorithms

Зогсоох бодлого
The Halting Problem

- Компьютерийн ухааны хамгийн алдартай теоремийн тухай энд товч авч үзэх болно.
- Ямарч процедур ашиглан шийдэж болохгүй тийм асуудал оршин байдаг.
- Ийм асуудлын нэг бол the **halting problem** юм.
- **Энэ бодлого**: **нэг ролт нь компьютер програм, нөгөө ролт нь энэ програм ролт байх ийм хос ролт бүхий процедур**, *хэрэв энэ програм тухайн ролтын хувьд хэзээ нэгэн цагт зогсож чадах уу гэсэн асуултад хариулт өгдөг бол* **ийм процедур оршин байхуу гэсэн асуултад хариулах явдал юм.**
- Хэрэв ийм процедур оршин байдагсан бол бидэнд их тустай байх байлаа.
- 1936 онд Alan Turing ийм процедур оршин байхгүй гэдгийг баталсан байдаг.
- Хэрэв програм “зогсвол” бид хүссэн үр дүнгээ гарган авч чадна. Хэрэв хангалттай хугацаа өнгөрөөд байхад програм зогсохгүй цааш үргэлжлэн ажилаад байвал бид уг програмыг хэзээ ч зогсохгүй эсвэл цааш хүлээх хэрэгтэй эсэхээ мэдэх боломжгүй юм.

- *H(P, I)* нь the halting problem-ийн шийд байдаг.
- *H(P, I) процедур хоёр ороллтой.*
  - *Нэг нь* програм *P,*
  - *харин нөгөө нь P програмын оролт I байг.*
- Хэрэв өгөгдсөн оролт *I*-ийн хувьд *P* програм зогсдог бол *H(P,I)* процедур "halt" мөрийг гаралт дээр бичиж өгнө.
- Эсрэг тохиолдолд, *H(P, I) процедур* "loops forever" мөрийг гаралт дээр бичиж өгнө.
- Эсрэгээс нь баталгаа хийе.
- Ямар нэг процедурыг кодолсон гэвэл тэр нь тэмдэгтүүдийн мөр хэлбэртэй байх бөгөөд уг мөрийг битүүдийн дараалал гэж үзэж болно.
- Энэ нь програмыг өөрийг нь өгөгдөл болгон ашиглаж болно гэсэн үг юм.
- Иймд програмыг өөр нэг програмын оролт болгон ашиглаж болно
- Тиймээс *P* нь *H*-ийн хоёр оролтын аль аль нь байж чадна.
- *P-ийн оролт P өөрөө (сору) байхад P програм зогсох эсэхийг H тодорхойлж чадах байх ёстой.*
- *K(P) гэсэн хялбар процедурыг байгуулъя. K(P) нь дараах байдлаар ажилладаг:*
  - *H(P,P)-ийн гаралтыг ашиглана.*
  - *H(P,P)-ийн гаралт* "loops forever" бол өөрөөр хэлбэл *P-ийн оролт P өөрөө үед P програм төгсгөлгүй давталт хийх бол K(P) зогсдог.*
  - *Хэрэв H(P,P) гаралт* "halt" бол өөрөөр хэлбэл *P-ийн оролт P өөрөө үед P програм зогсдог бол K(P) төгсгөлгүй давталт хийдэг* байг.
- Өөрөөр хэлбэл, *K(P) нь H(P,P)-ийн гаралт ямар байна түүний эсрэг ажилладаг байг.*
- Одоо *K-ийн оролт K өөрөө байсан гэж үзье.*
- *Хэрэв H(K,K)-ийн гаралт* "loops forever" бол *K-ийн тодорхойлолт ёсоор K(K)* зогсоно.
- Эсрэг тохиолдолд, хэрэв *H(K,K)-ийн гаралт* "halt" бол *K-ийн тодорхойлолт ёсоор K(K)* төгсгөлгүй давталт хийнэ.
- Аль ч тохиолдолд, *H(K,K)*-ийн гаралттай зөрчилд хүрнэ.
- Иймд, *H үргэжл зөв хариулт өгч чадахгүй*. Өөрөөр хэлбэл, the halting problem-ийг шийдэх ямар ч процедур оршин байхгүй.

The halting problem



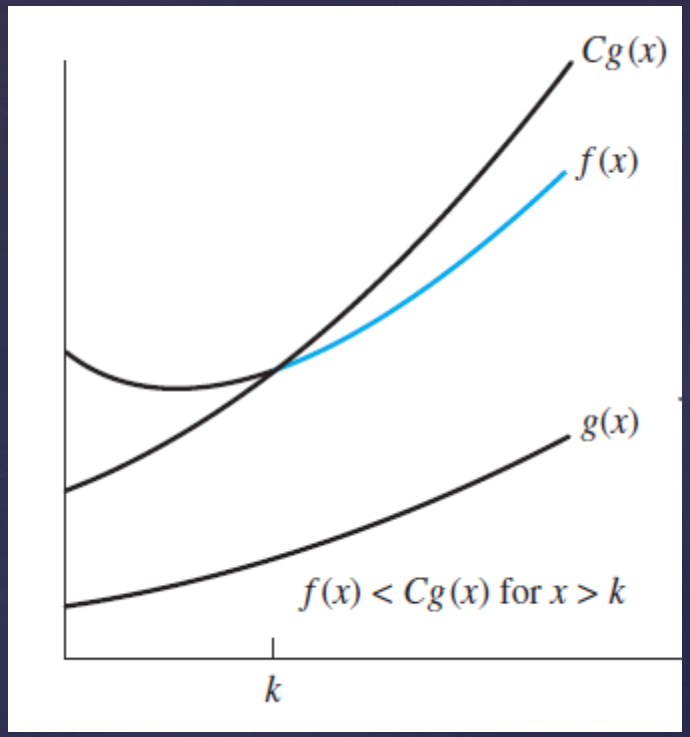
The halting problem

Функцийн өсөлт

- Функцийн өсөлтийг тодорхойлохдоо тусгай тэмдэглээ ашигладаг.
- f ба g функцүүд нь бүхэл тоон олонлог эсвэл бодит тоон олонлог дээр тодорхойлогдох ба бодит тоон олонлогт утгаа авдаг байг. Хэрэв C ба k тогтмолууд олдоод $x > k$ байх бүх x-үүдийн хувьд

$$|f(x)| \leq C|g(x)|$$

  бол $f(x)$ нь $O(g(x))$ гэнэ.
- Үүнийг "$f(x)$ нь том-О $g(x)$" гэж уншина.
- $g(x)$-ийг ямар нэгэн тогтмолоо үржүүлсэн үржвэрээс $f(x)$ нь удаан өсөлттэй гэдгийг харуулж байна.



Функцийн өсөлт

Жишээ

$f(x) = x^2 + 2x + 1$ нь $O(x^2)$ болохыг үзүүл.

- $x > 1$ үед $x < x^2$ ба $1 < x^2$ байна.
- Иймд, $0 \leq x^2 + 2x + 1 \leq x^2 + 2x^2 + x^2 = 4x^2$
- Эндээс, $C = 4$ ба $k = 1$ буюу $f(x)$ нь $O(x^2)$.

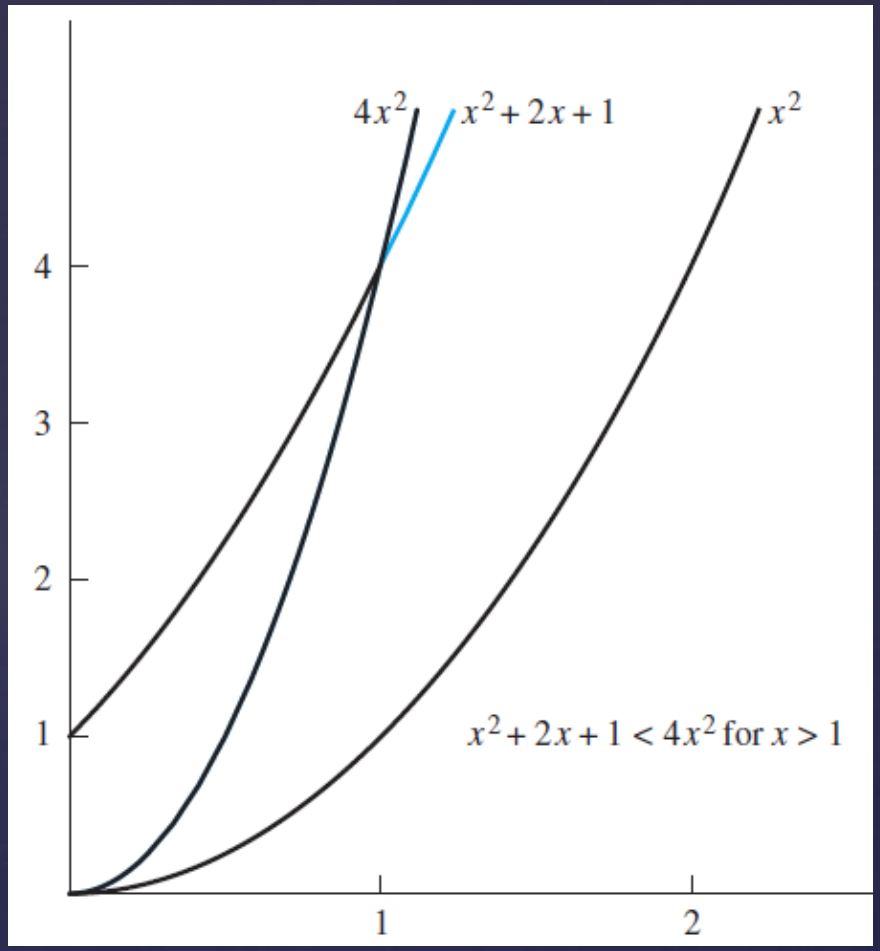
Үүний адилаар $f(x)$ функцийг $x > 2$ үед үнэлж болно.

- $x > 2$ байг. Тэгвэл $2x \leq x^2$ ба $1 \leq x^2$ байна.
- Иймд, хэрэв $x > 2$ бол

  $0 \leq x^2 + 2x + 1 \leq x^2 + x^2 + x^2 = 3x^2$

  байна.
- Эндээс, $C = 3$ ба $k = 2$ буюу $f(x)$ нь $O(x^2)$.

"$f(x)$ нь $O(x^2)$" тэмдэглээнээс x^2 функцийг x^2-аас их утгатай дурын функцээр сольж болох нь харагдаж байна. Жишээлбэл, $f(x)$ нь $O(x^3)$, $f(x)$ нь $O(x^2 + x + 7)$, гэх мэт.



Функцийн өсөлт

- $f(x)$ нь $O(g(x))$ бол зарим тохиолдолд $f(x) = O(g(x))$ гэж бичих нь бий гэвч энд буй = тэмдэг жинхэнэ утгаар тэнцүү гэсэн санааг илэрхийлэхгүй.
- Хэдий тийм боловч $f(x) \in O(g(x))$ тэмдэглээг хэрэглэх нь илүү тохиромжтой бөгөөд учир нь $O(g(x))$ тэмдэглээ үнэн хэрэгтээ функцүүдийн олонлогийг илэрхийлж байгаа юм.
- $O(g(x))$ тэмдэглээнд буй $g(x)$ функцийг аль болох жижиг функцээр сонгох ба x^n хэлбэрийн функцийг сонгох нь элбэг

Функцийн өсөлт

- $f(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0$, бөгөөд энд, $a_0, a_1, \ldots, a_{n-1}, a_n$ тоонууд нь бодит тоонууд бол $f(x)$ нь $O(x^n)$. Энд, $C = |a_0| + |a_1| + \cdots + |a_n|$ ба $k = 1$ байна.
- Эхний n ширхэг эерэг бүхэл тоонуудын нийлбэр

$$1 + 2 + \cdots + n \leq n + n + \cdots + n = n^2$$

буюу $1 + 2 + \cdots + n$ нь $O(n^2)$ байна. Энд, $C = 1$ ба $k = 1$ байна.

- $f(n) = n!$ байг. Тэгвэл

$$\begin{aligned} n! &= 1 \cdot 2 \cdot 3 \cdot \cdots \cdot \mathrm{n} \\ &\leq n \cdot \mathrm{n} \cdot \mathrm{n} \cdot \cdots \cdot n \\ &= n^n \end{aligned}$$

буюу $n!$ нь $O(n^n)$ байна. Энд, $C = 1$ ба $k = 1$ байна.

- Хоёр талаас нь логарифм авбал,

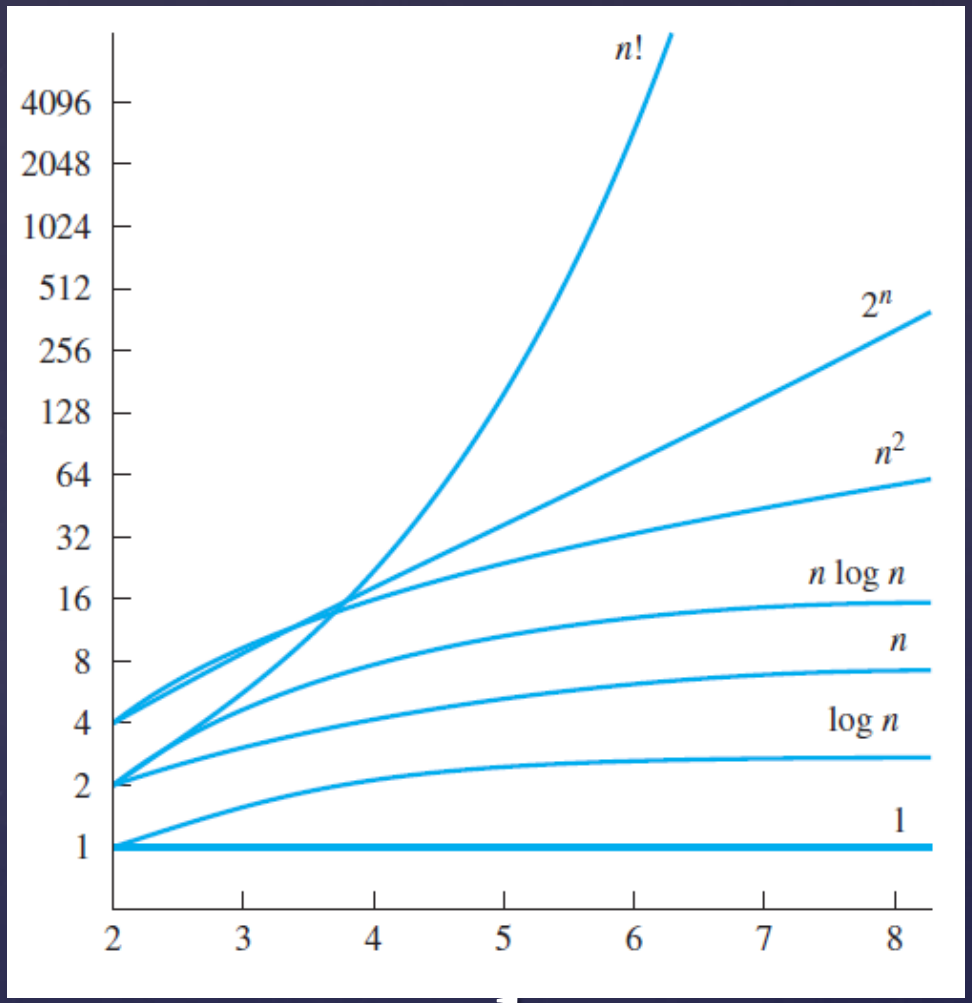
$$\log n! \leq \log n^n = n \log n$$

буюу $\log n!$ нь $O(n \log n)$ байна. Энд, $C = 1$ ба $k = 1$ байна.

Зарим чухал функцүүдийн том-О үнэлгээ

- $n < 2^n$ тэнцэл бишийг ашиглавал n нь $O(2^n)$ гэсэн дүгнэлтэд хүрэх ба энд, $k = C = 1$ байна.
- $n < 2^n$ тэнцэл бишийн хоёр талаас 2 суурьтай логарифм авбал $\log n < n$ тэнцэлбиш гарч ирэх бөгөөд эндээс $\log n$ нь $O(n)$ байх ба дахиад $C = k = 1$ байна.
- Хэрэв бидэнд b суурьтай логарифм байсан бол $\log_b n$ нь $O(n)$ байна. Учир нь, $\log_b n = \frac{\log n}{\log b} < \frac{n}{\log b}$. Энд, $C = \frac{1}{\log b}$ ба $k = 1$ байна.

Зарим чухал функцүүдийн том-О үнэлгээ



Зарим чухал функцүүдийн том-О үнэлгээ

ШУТИС, Хэрэглээний шинжлэх ухааны сургууль, Я.Лутбат

Алгоритмын комплекс чанар

Алгоритмын комплекс чанар

- Эрчимтэй алгоритмын тухай энэ хичээл дээр товч авч үзэх болно.
- Алгоритмын эрчимтэй байдлыг (эрчмийг) хэрхэн шинжлэх вэ?
- Өгөгдсөн хэмжээ бүхий бодлогыг тухайн алгоритмаар компьютерийн тусламжтай шийдэхэд шаардагдах нийт хугацаагаар алгоритмын эрчмийг хэмжидэг. Ийм шинжилгээг алгоритмын *хугацааны комплекс чанар (*товчдоо, *хугацааны комплекс)* гэнэ.
- Алгоритмын эрчмийг хэмжих өөр нэг арга бол өгөгдсөн хэмжээ бүхий бодлогыг тухайн алгоритмаар компьютерийн тусламжтай шийдэхэд шаардагдах нийт санах ойн хэмжээг ашиглах явдал юм. Харин энэ төрлийн шинжилгээг алгоритмын *огторгуйн комплекс чанар (огторгуйн комплекс)* гэнэ.

Хугацааны комплекс чанар

- Өгөгдсөн хэмжээ бүхий бодлогыг шийдэхэд тухайн алгоритмд шаардагдах нийт үйлдлийн тоогоор алгоритмын хугацааны комплекс чанар илэрхийлэгдэнэ.
- Хугацааны комплекс чанарыг тодорхойлох үйлдлийн тоо нь компьютерийн бодит хугацааг тооцохгүй. Учир нь, алгоритмд хийгдэх үндсэн үйлдлүүдэд шаардагдах хугацаа ялгаатай компьютеруудад харилцан адилгүй.
- Тодруулбал, өндөр хурдны компьютерууд битүүд дээрх үндсэн үйлдлүүдийг (жишээлбэл, нийлбэр, үржвэр, жиших гэх мэт) 10^{-11}секундэд (10 пикосекунд), харин PC компьютерууд 10^{-8} секунд (10 наносекунд) гүйцэтгэдэг.
- Algorithm 1-ыг ашиглан хэрхэн хугацааны комплекс чанарыг шинжлэх вэ гэдгийг харуулъя.

Жишээ

- Жишээнд, харьцуулах үйлдлийн тоогоор алгоритмын хугацааны комплекс чанарыг хэмжинэ. Учир нь, энд харьцуулах үйлдэл бол алгоритм хийгдэж буй үндсэн үйлдэл юм.
- Дурын дарааллаар жагсаан бичсэн n элемент бүхий олонлогоос максимум элементийг олохдоо максимум элементэд жагсаалтын эхний элементийг сонгон авна.
- Улмаар, жагсаалт дууссан эсэхийг харуулах $i \leq n$ харьцуулалт хийгдсэний дараа түр сонгон авсан максимумыг хоёр дахь элементтэй харьцуулна. Хэрэв хоёр дахь элемент максимумаас их бол хоёр дах элементийг максимумаар сонгоно.
- Энэ процедурыг жагсаалтын элемент бүрийн хувьд (2 харьцуулах үйлдэлтэй) цааш үргэлжлүүлнэ. Эхний харьцуулах үйлдэл нь $i \leq n$ бол дараагийн харьцуулах үйлдэл нь $max < a_i$ юм.
  - Харьцуулах үйлдлүүдийг жагсаалтын 2 дах элементээс n дүгээр элемент хүртэл давтах ба нэмэлт нэг харьцуулалтыг давталтаас гарахын тулд $i = n + 1$ үед хийнэ. Иймд нийт $2(n-1)+1 = 2n-1$ тооны харьцуулах үйлдэл хийгдэнэ.
- Тэгвэл, *n* элемент бүхий олонлогоос максимум элементийг олох алгоритмын хугацааны комплекс харьцуулах үйлдлийн тоогоороо $\Theta(n)$ байна.

- Algortihm 2-ийн хугацааны комплексийг тодорхойл.
- Харьцуулах үйлдлийн тоогоор хугацааны комплексийг тодорхойлъё.
- Давталтын алхам бүрт 2 харьцуулах үйлдэл хийгдэнэ. Нэг нь, жагсаалт дууссан эсэхийг тодохойлох $i \leq n$ харьцуулалт, нөгөө нь, x-ийг жагсаалтын i дүгээр элементтэй жиших $x \neq a_i$ харьцуулалт юм.
- Эцэст нь, $i \leq n$ харьцуулалтыг дахин нэг хийнэ.
- Иймд, хэрэв $x = a_i$, $2i + 1$ тооны харьцуулалт хийгдэнэ.
- Хэрэв жагсаалтанд x элемент байхгүй бол $2n + 2$ тооны харьцуулах үйлдэл хийгдэнэ.
  - x-ийг $a_i, i = 1,2, \dots, n$ тоотой тэнцэх эсэхийг шалгах $2n$ харьцуулалт хийгдэх ба нэмэлт нэг харьцуулалт давталтаас гарахад хийгдэнэ. Харин дахин нэг харьцуулалт давталтын гадна хийгдэнэ
  - Иймээс, x нь жагсаалтад байхгүй бол нийт $2n + 2$ тооны харьцуулах үйлдэл хийгдэнэ.
- Иймд шугаман хайлтын хувьд *хамгийн муу тохиолдолд* $\Theta(n)$ харьцуулалт хийнэ.

Хамгийн муу тохиолдол

- Өмнөх жишээнд бид хамгийн муу тохиолдолд алгоритм хийгдэх үйлдлийн тоонд шинжилгээ хийсэн
- Хамгийн муу тохиолдолд шинжилгээгээр тухайн алгоритмаар өгөгдсөн хэмжээ бүхий бодлогыг шийдэхэд шаардагдах үйлдлийн тоо хамгийн их утгандаа хүрдэг.
- Нөгөө талаас хамгийн муу тохиолдол нь тухайн алгоритм ямар нэг шийд гаргасан байхын тулд хийх үйлдлийн тоонд баталгаа өгч байгаа хэрэг юм.

Дундаж тохиолдол

- Өөр нэг чухал шинжилгээний төрөл бол дундаж тохиолдол юм.
- Дундаж тохиолдолд шинжилгээгээр тухайн алгоритмаар бүхий боломжит хэмжээ бүхий бодлогыг шийдэхэд шаардагдах үйлдлийн тооны дундаж утга тодорхойлогдоно.
- Дундаж тохиолдолд шинжилгээ хийх нь хамгийн муу тохиолдолд хийх шинжилгээнээс илүү төвөгтэй байдаг.